

Agrownex - AI System for Next-Gen Weather Insurance Pricing

Submitted Towards the Partial Fulfilment of the Requirements for the Award of
the Degree of

Bachelor of Technology

By

Shakti Prosad Dey (23124100)

Mohmmad Aslam (23124065)

Under the Mentorship of

Dr. Vijay Kumar
Assistant Professor



Department of Information Technology

Dr B R Ambedkar National Institute of
Technology Jalandhar-144008, Punjab (INDIA)

Undertaking

We declare that the project work presented in this report entitled **Agrownex - AI System for Next-Gen Weather Insurance Pricing**, submitted to the Department of Information Technology, Dr. B. R. Ambedkar National Institute of Technology, Jalandhar, for the award of the Bachelor of Technology degree in Bachelor of Technology in Information Technology, is our original work. We have not plagiarized or submitted the same work for the award of any other degree. In case this undertaking is found incorrect, we accept that our degree may be unconditionally withdrawn.

Shakti Prosad Dey (23124100)

Mohmmad Aslam (23124065)

Department of Information Technology

Dr B R Ambedkar National Institute of Technology Jalandhar

Punjab, India

[25 March, 2026]

Certificate

This is to certify that the project report entitled **Agrownex - AI System for Next-Gen Weather Insurance Pricing** submitted by Shakti Prosad Dey (23124100), Mohmmad Aslam (23124065), to Dr B R Ambedkar National Institute of Technology Jalandhar, in partial fulfillment for the award of the degree of B. Tech in Information Technology has been carried out under our supervision and that this work has not been submitted elsewhere for a degree.

Dr. Naveen Kumar Gupta

Assistant Professor

Department of Information Technology

Dr B R Ambedkar National Institute of Technology,

Jalandhar Punjab ,INDIA

25 March, 2026

Acknowledgement

We would like to express our sincere gratitude to all those who have contributed to the successful completion of this project.

First and foremost, we extend our heartfelt thanks to our project supervisor, Dr. Vijay Kumar Chahar , for their invaluable guidance, continuous support, and encouragement throughout the duration of this project. Their expertise and constructive feedback have been instrumental in shaping this work. We are deeply grateful to Dr. Vijay Kumar Chahar, Head of the Department of Information Technology, for providing us with the necessary facilities and resources to carry out this project. We would also like to thank all the faculty members of the Department of Information Technology for their support and valuable suggestions during various stages of this project. Our sincere thanks go to our family and friends for their constant encouragement and support throughout our academic journey.

Finally, we acknowledge the contributions of all those who directly or indirectly helped us in completing this project successfully.

Shakti Prosad Dey (23124100)

Mohmmad Aslam (23124065)

Abstract

This project presents **Agrownex**, an AI-powered platform designed to modernize and automate weather-based agricultural insurance pricing. With increasing climate unpredictability, traditional insurance models have become inefficient, slow, and often inaccurate for farmers. There is a growing need for intelligent systems that can dynamically assess risk and provide transparent, real-time insurance solutions tailored to local conditions..

The proposed system addresses this challenge by integrating **hyperlocal weather analytics**, agricultural risk parameters, and advanced machine learning techniques to deliver accurate and adaptive insurance pricing. Agrownex combines multiple components to enhance usability and decision-making, including an intelligent AI chat assistant for user interaction, dynamic insurance quote generation based on real-time inputs, and an interactive weather dashboard that visualizes environmental conditions affecting crop risk..

The core of the system lies in its predictive modeling capabilities. Various machine learning algorithms were evaluated, among which **XGBoost** emerged as the best-performing model for insurance risk prediction and pricing. It demonstrated high accuracy, robustness, and reliability in handling complex, non-linear relationships between weather patterns and agricultural outcomes.

This work provides a scalable and intelligent framework for next-generation agri-insurance systems and lays the groundwork for future enhancements such as real-time claim automation, integration with IoT-based farm sensors, and expansion to multi-regional risk modeling..

Keywords: Agricultural Insurance, XGBoost, Weather Analytics, Machine Learning, AI Chatbot

Contents

Acknowledgement	iii
Abstract	iv
1 Introduction	1
1.1 Overview.....	1
1.2 Problem Statement.....	1
1.3 Motivation.....	1
1.4 Objectives	1
1.5 Scope of the Project	1
1.6 Organization of Report	1
2 Literature Review	2
2.1 Introduction.....	2
2.2 Existing Systems/Approaches.....	2
2.2.1 Traditional Approaches	2
2.2.2 Modern Solutions	2
2.3 Technology Review	3
2.3.1 Technology/Method 1	3
2.3.2 Technology/Method 2	3
2.4 Comparative Analysis.....	3
2.5 Research Gaps.....	3
2.6 Summary.....	3
3 Background and Preliminaries	4
3.1 Introduction.....	4
3.2 Fundamental Concepts.....	4
3.2.1 Concept 1.....	4
3.2.2 Concept 2.....	4
3.3 Technologies and Tools	5
3.3.1 Technology Stack.....	5
3.3.2 Development Environment.....	5
3.4 Theoretical Framework.....	5
3.5 Summary.....	5
4 Proposed Methodology	6
4.1 Introduction.....	6
4.2 System Overview	6
4.3 System Architecture.....	6

4.3.1	Architecture Design.....	6
4.3.2	Component Description.....	6
4.4	Module Description	7
4.4.1	Module 1: Data Acquisition Module.....	7
4.4.2	Module 2:Preprocessing & Feature Extraction Module.....	7
4.5	Algorithms and Techniques	7
4.5.1	Algorithm 1	7
4.6	Design Diagrams.....	8
4.6.1	Use Case Diagram.....	8
4.6.2	Class Diagram	9
4.6.3	State Activity Diagram.....	10
4.6.4	Workflow	11
4.6.5	Homepage.....	11
4.7	Database Design	12
4.8	User Interface Design	12
4.9	Security and Privacy	12
4.10	Implementation Strategy.....	12
4.11	Summary.....	12
5	Experimental Results and Analysis	13
5.1	Introduction.....	13
5.2	Experimental Setup.....	13
5.2.1	Hardware Configuration.....	13
5.2.2	Software Configuration	13
5.3	Implementation Details.....	13
5.3.1	Feature Implementation.....	13
5.4	Testing and Validation.....	14
5.4.1	Unit Testing.....	14
5.4.2	Integration Testing	14

5.4.3	System Testing	14
5.4.4	User Acceptance Testing	14
5.5	Results	14
5.5.1	Performance Metrics	14
5.5.2	Functional Results	14
5.5.3	UI for Agrownex	14
5.6	Analysis and Discussion	15
5.6.1	Key Findings	15
5.6.2	Advantages	16
5.6.3	Limitations	18
5.7	Summary	19
6	Conclusion and Future Work	20
6.1	Conclusion	20
6.1.1	Summary of Work	20
6.1.2	Key Contributions	20
6.1.3	Achievement of Objectives	20
6.1.4	Impact and Significance	20
6.2	Future Scope	21
6.2.1	Short-term Enhancements	21
6.2.2	Long-term Vision	21
6.2.3	Research Directions	21
6.3	Final Remarks	21
	References	22
A	Source Code Snippets	23

Chapter 1

Introduction

1.1 Overview

This project introduces **Agrownex**, an AI-powered platform that modernizes agricultural insurance through the integration of real-time weather data, advanced machine learning models, and automated risk analysis. By leveraging hyperlocal weather analytics and intelligent algorithms, Agrownex enables instant and precise insurance pricing tailored to specific geographic and environmental conditions. The system aims to replace outdated manual workflows with a data-driven, scalable, and transparent approach to insurance decision-making..

1.2 Problem Statement

How can we accurately predict agricultural insurance risk and pricing using real-time hyperlocal weather data and machine learning models, while accounting for climate variability, location-specific factors, and uncertainty in environmental conditions?

1.3 Motivation

The proposed system aims to eliminate the inefficiencies of traditional agricultural insurance methods, which rely on manual assessment and delayed processing. Instead, it leverages real-time weather data and machine learning models to enable automated and accurate risk prediction without the need for manual intervention. Furthermore, the focus is on building robust models that can adapt to varying climate conditions and location-specific factors, ensuring reliable performance and making the system suitable for real-world deployment..

1.4 Objectives

1. Analyze the impact of hyperlocal weather parameters (temperature, rainfall, humidity) on agricultural risk and insurance pricing.
2. Collect and preprocess real-time and historical weather datasets for accurate model training.
3. Implement data preprocessing and feature engineering to handle missing values, anomalies.
4. Train and evaluate machine learning models (e.g., XGBoost) for risk prediction and automated pricing

1.5 Scope of the Project

The project targets agricultural insurance risk prediction using real-time and historical weather data, along with machine learning models for pricing estimation. It includes data collection from weather sources, preprocessing, feature engineering, model training, and evaluation. The system does not focus on full claim settlement or direct integration with insurance providers.

Chapter 2

Literature Review

2.1 Introduction

The purpose of this literature review is to analyze and compare existing research, technologies, and methods used in agricultural risk assessment and insurance systems. This section reviews traditional insurance approaches based on manual evaluation and historical data, along with modern data-driven solutions that use weather analytics and machine learning. The review also identifies gaps in current methods and demonstrates how Agrownex addresses these challenges..

2.2 Existing Systems/Approaches

Agricultural insurance and risk prediction have been implemented using various approaches, including manual assessment, statistical models, and data-driven systems. Each method differs in accuracy, speed, scalability, and usability..

2.2.1 Traditional Approaches

Traditional agricultural insurance systems mainly rely on manual inspection, historical yield data, and generalized climate assumptions. Field agents assess crop damage and estimate risk based on past trends. While these methods are widely used, they are slow, prone to human error, and often lack precision.

Another approach involves rule-based or statistical models, which use predefined formulas to calculate insurance premiums. However, these models fail to capture real-time environmental variations and complex relationships between weather and crop yield.

Overall, traditional systems are inefficient, time-consuming, and lack scalability. They depend heavily on manual processes, leading to delayed payouts and reduced trust among farmers.

2.2.2 Modern Solutions

Modern agricultural systems focus on data-driven and automated approaches using weather analytics, IoT, and machine learning. One approach involves using satellite data and weather APIs to monitor environmental conditions and estimate crop risk. These systems provide better accuracy than traditional methods but may still lack fine-grained local insights.

A more advanced approach uses machine learning models to analyze large datasets and predict agricultural risks. Algorithms such as regression models, decision trees, and ensemble methods (e.g., XGBoost) are used to improve prediction accuracy.

Additionally, AI-based platforms integrate data analytics with user interfaces such as dashboards and chatbots, making systems more interactive and accessible. Overall, modern solutions improve efficiency, reduce manual effort, and enable scalable deployment for real-world applications.

2.3 Technology Review

Weather data analytics plays a key role in agricultural risk prediction. Real-time and historical data such as temperature, rainfall, humidity, and soil conditions are collected from APIs and databases. These parameters are used to understand environmental patterns and their impact on crop yield.

2.3.1 Technology/Method 1

Weather data analytics plays a key role in agricultural risk prediction. Real-time and historical data such as temperature, rainfall, humidity, and soil conditions are collected from APIs and databases. These parameters are used to understand environmental patterns and their impact on crop yield.

2.3.2 Technology/Method 2

Machine learning techniques are used to predict risk and automate insurance pricing. Common algorithms include:

- Lightgbm
- Random Forest
- XG Boost
- SVM

2.4 Research Gaps

Despite advancements in agricultural technology, several challenges still exist. Many systems lack real-time risk prediction and depend on outdated or aggregated data. There is limited use of hyperlocal weather analytics, which reduces prediction accuracy.

Traditional insurance systems are still slow, manual, and not easily accessible to farmers. Additionally, there is a lack of intelligent user interfaces that can guide farmers in understanding insurance policies and risk levels. This project addresses these gaps by developing an AI-powered platform that combines real-time weather data, machine learning models, and interactive tools for accurate and transparent insurance pricing.

2.5 Summary

This chapter reviewed existing agricultural insurance approaches, from traditional manual systems to modern data-driven solutions. The comparison highlights the advantages of using machine learning and real-time weather analytics for improving accuracy, efficiency, and scalability. The identified research gaps support the need for an intelligent system like Agrownex, which focuses on automated risk prediction and insurance pricing.

Chapter 3

Background

3.1 Introduction

This chapter provides the essential background required to understand the development of the Agrownex system for agricultural insurance risk prediction. It explains the foundational concepts behind weather-based analytics, risk modeling, and machine learning techniques used for insurance pricing. The chapter also introduces the technologies, tools, and software environment used in the project along with the theoretical concepts that support system implementation.

3.2 Fundamental Concepts

Agricultural risk prediction involves analyzing environmental factors such as weather conditions and their impact on crop yield. In this project, real-time and historical weather data are used to assess risk levels and generate insurance pricing.

3.2.1 Concept 1 – Weather-Based Risk Analysis

Weather-based risk analysis focuses on evaluating how environmental parameters like temperature, rainfall, humidity, and extreme events affect agricultural output. Variations in these parameters directly influence crop health and yield.

3.2.2 Concept 2 – Machine Learning-based Classification

Machine learning is used to identify patterns between weather conditions and agricultural risk. After preprocessing and feature engineering, models are trained to predict risk scores and insurance premiums.

Common algorithms used:

- Linear Regression – Basic prediction model.
- Decision Trees – Handles non-linear relationships.
- Random Forest – Improves accuracy using ensemble learning.
- XGBoost – Provides high performance and robustness

Process

1. Data Collection (weather datasets)
2. Data Cleaning & Preprocessing
3. Feature Extraction
4. Model Training
5. Activity Prediction

3.3 Technologies and Tools

The system uses a combination of software tools and technologies for data collection, processing, and prediction. Weather data is collected using APIs and external datasets. The implementation is carried out using Python due to its strong support for data analysis and machine learning.

3.3.1 Technology Stack

The frontend includes a web-based interface that provides an interactive dashboard for weather insights and insurance details, along with an AI chatbot for user interaction.

The backend is implemented in Python and handles data processing, model execution, and API integration. Machine learning models are developed using libraries such as Scikit-learn and XGBoost.

For data storage, datasets and results can be stored using CSV files or databases like MySQL for better scalability and management.

3.3.2 Development Environment

The system is developed using Python (version 3.10 or above) with tools such as Jupyter Notebook, VS Code, or PyCharm. Version control is managed using Git and GitHub. Documentation is prepared using standard tools such as Microsoft Word..

The project runs on general-purpose computing systems and does not require specialized hardware, making it easy to deploy and scale..

3.4 Theoretical Framework

The theoretical foundation of this project is based on data analytics, machine learning, and environmental modeling. Weather patterns are analyzed using statistical and predictive techniques to understand their impact on agriculture.

Machine learning models are applied using supervised learning methods to map weather inputs to risk outputs. Feature engineering and data preprocessing techniques are used to improve model performance.

The system is evaluated using performance metrics such as accuracy, error rate, and prediction reliability to ensure effective risk estimation and pricing.

3.5 Summary

This chapter introduced the core concepts, technologies, tools, and theoretical foundations required for developing the Agrownex system. Weather analytics and machine learning form the backbone of the platform. These elements support the implementation of an intelligent system for automated agricultural risk prediction and insurance pricing.

Chapter 4

Proposed Methodology

4.1 Introduction

This chapter explains the proposed methodology for developing the Agrownex system for agricultural insurance risk prediction and pricing. The system aims to estimate crop risk and generate accurate insurance quotes by analyzing real-time and historical weather data. The approach focuses on data-driven automation, ensuring faster, reliable, and scalable solutions using machine learning techniques.

4.2 System Overview

The proposed system collects weather data from various sources and processes it through a structured pipeline to predict agricultural risk. Initially, raw data such as temperature, rainfall, and humidity is gathered using APIs or datasets. This data is then preprocessed to remove noise, handle missing values, and improve quality for analysis.

After preprocessing, relevant features are extracted to capture patterns between environmental conditions and crop risk. These features are used to train machine learning models, enabling the system to learn and predict insurance risk accurately.

Once trained, the model can generate risk scores and insurance pricing for new inputs, which are displayed through a dashboard or chatbot interface. This pipeline ensures efficient, real-time, and automated insurance estimation.

4.3 System Architecture

The system architecture is divided into three main phases: Data Collection, Data Processing, and Machine Learning Prediction. **Data Collection Phase:** Weather data is collected from external APIs or datasets. **Data Processing Phase:** The data undergoes preprocessing, cleaning, and feature extraction to prepare it for modeling. **Prediction Phase:** Machine learning models analyze the processed data and generate risk scores and insurance pricing.

4.3.1 Architecture Design

The system consists of multiple interconnected components. Weather APIs provide real-time environmental data, which is stored as raw input. This data is passed to the preprocessing unit, where cleaning and normalization are performed..

Next, the feature engineering unit extracts meaningful attributes from the data. These features are fed into the machine learning model (e.g., XGBoost), which predicts risk and pricing. Finally, the results are displayed through a user interface such as a dashboard or chatbot, providing clear and interactive outputs.

4.3.2 Component Description

The system consists of multiple functional components working together for accurate risk prediction and insurance pricing. The Data Collection Unit gathers real-time and historical weather data, which is then processed by the Preprocessing Unit to clean, normalize, and remove inconsistencies. The Feature Engineering Unit extracts meaningful features from the data, which are used by the Prediction Engine to estimate agricultural risk and insurance pricing using machine learning models. Finally, the User Interface

presents the results through an interactive dashboard and chatbot for easy user interaction.

4.4 Module Description

4.4.1 Module 1: Data Acquisition Module

This module collects weather data that influences agricultural risk. It captures parameters such as temperature, rainfall, and humidity from APIs or datasets.

The collected data is stored in structured formats like CSV or databases, making it ready for further processing and analysis.

4.4.2 Module 2: Preprocessing & Feature Extraction Module

This module processes raw weather data to make it suitable for machine learning models. It handles missing values, removes outliers, and normalizes the data..

After cleaning, it extracts meaningful features such as averages, trends, and seasonal patterns. These features help the model understand relationships between weather conditions and agricultural risk, improving prediction accuracy. For implementation, this module is developed in Python using libraries such as NumPy, SciPy, Pandas, and other feature engineering tools, enabling efficient data processing and feature extraction.

4.5 Design Diagrams

4.5.1 Use-Case Diagram

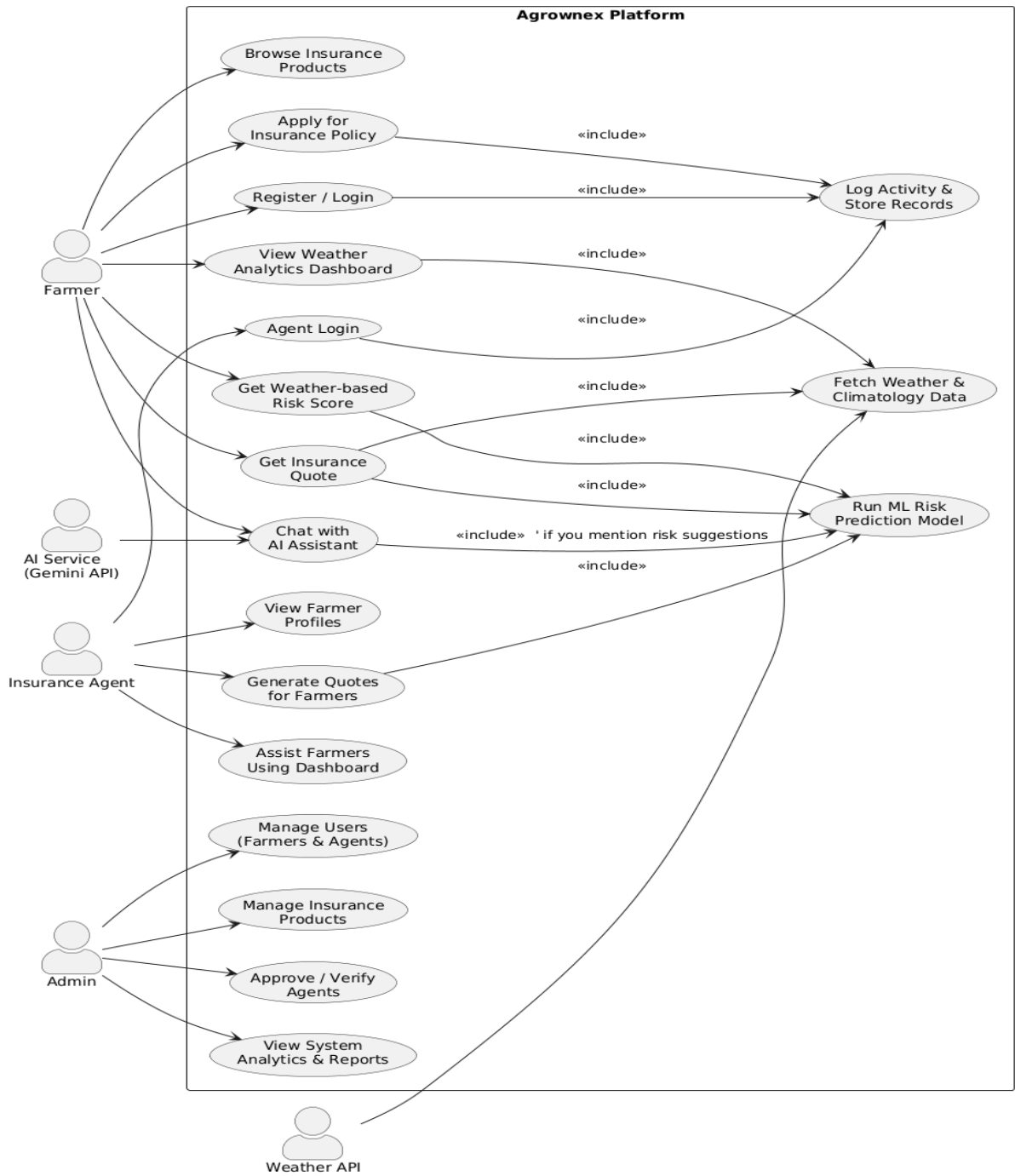


Figure 2.1: Use-Case Diagram

4.5.2 Class Diagram

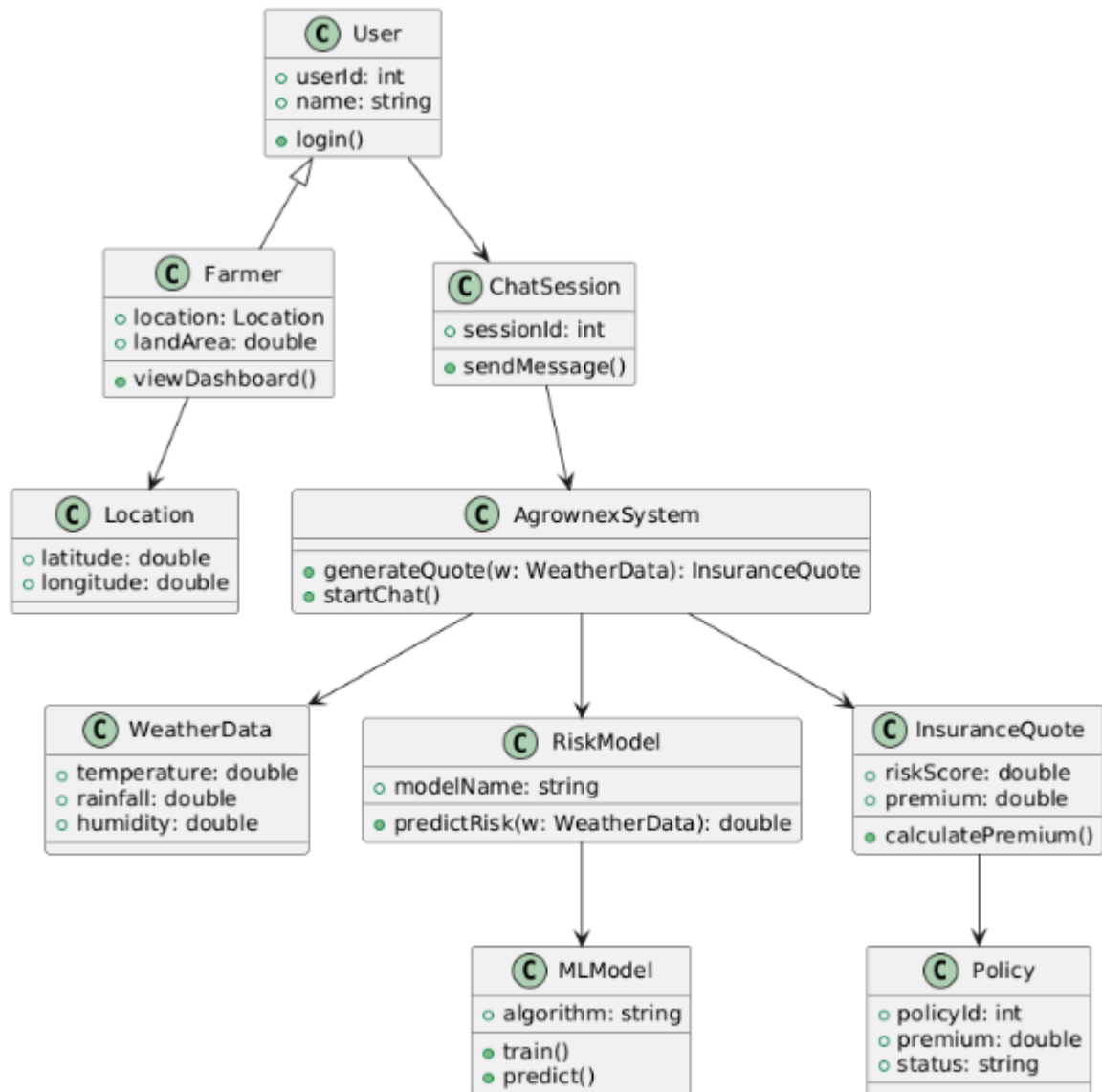
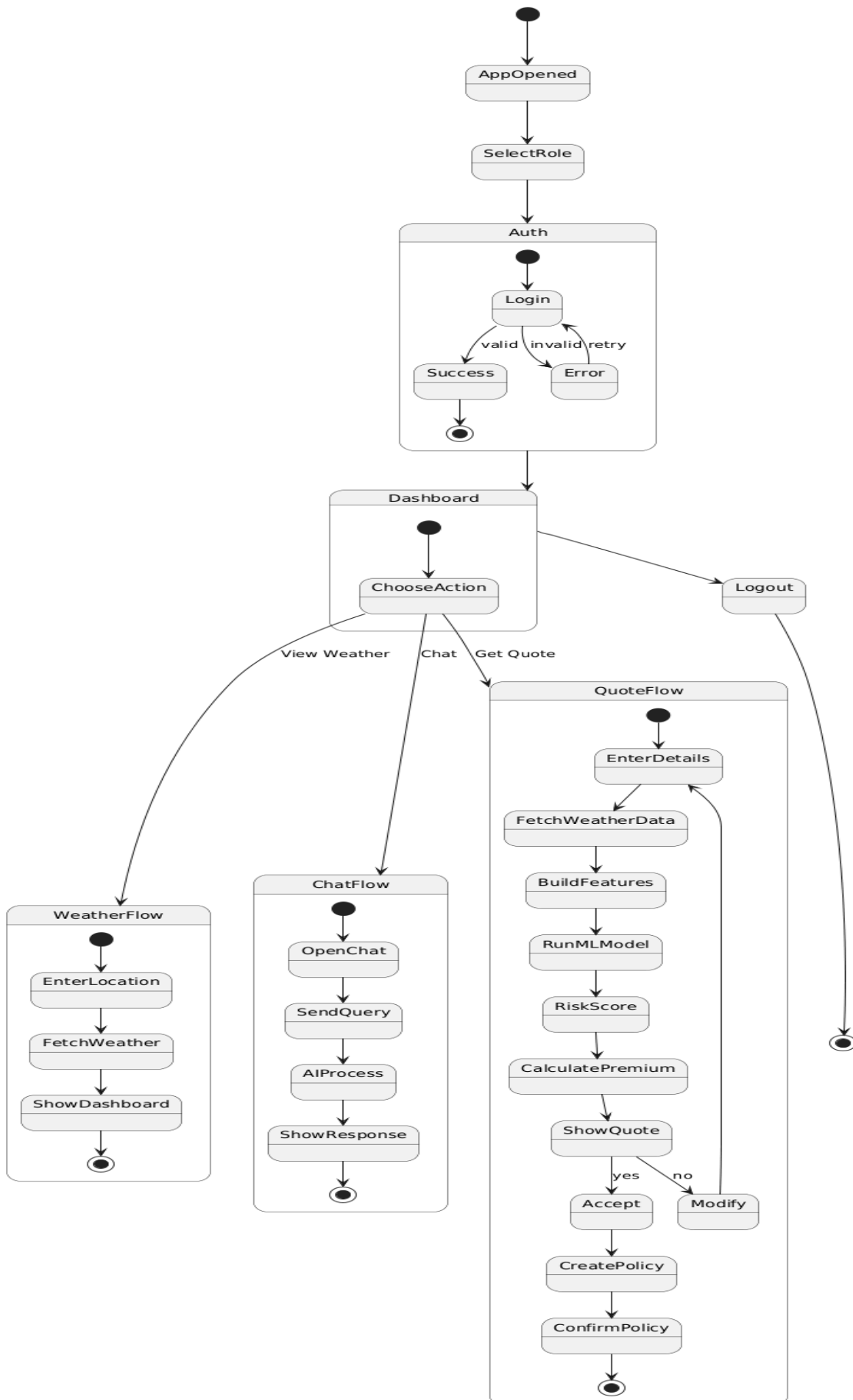


Figure 4.6.2: Class Diagram

4.5.3 State Activity Diagram



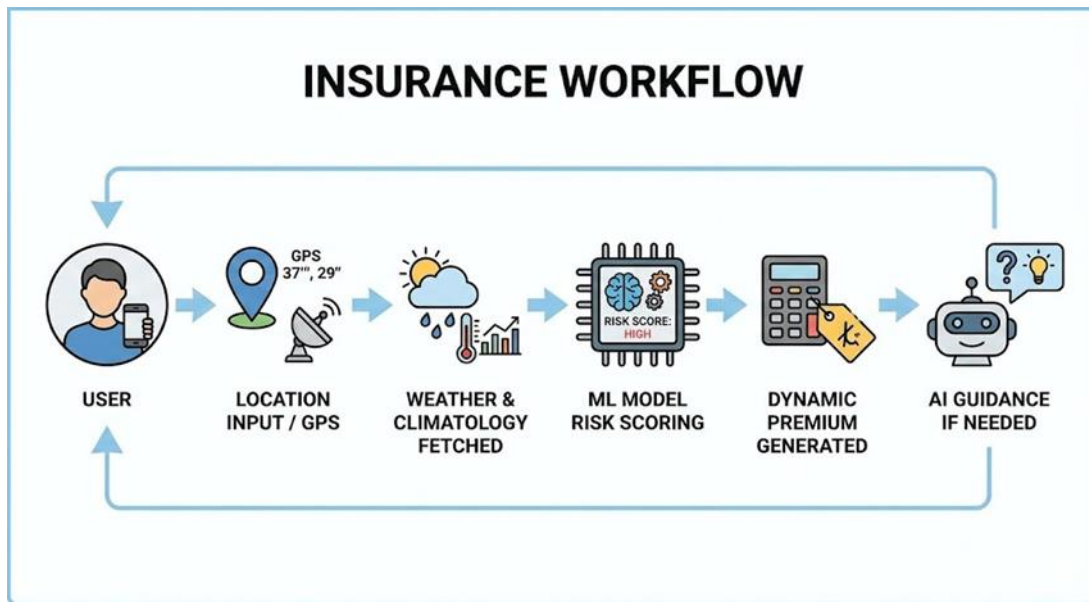


Figure 4.6.3: Workflow

4.5.4 Homepage

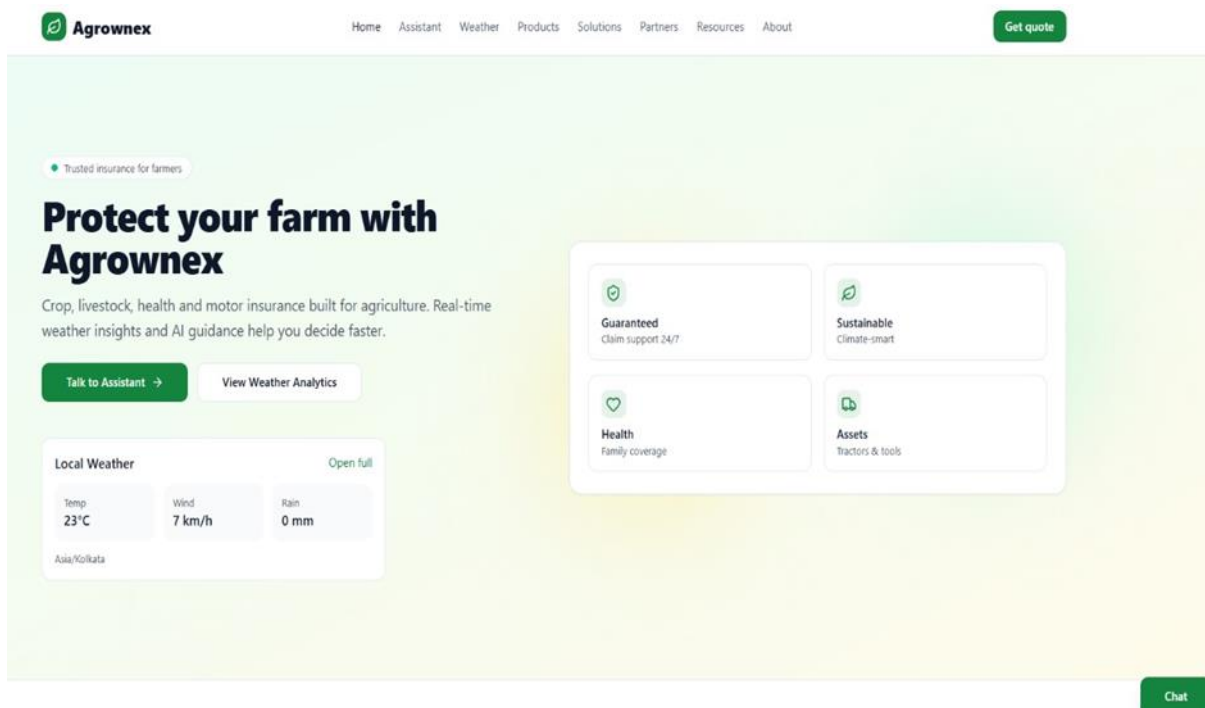


Figure 4.6.3: Homepage

4.6 User Interface Design

The user interface is designed to be simple, intuitive, and user-friendly, ensuring easy interaction for farmers. It can be implemented as a web-based application for better accessibility across devices.

The interface includes a dashboard that displays weather analytics, risk scores, and insurance details clearly to the user. In addition, visual components such as charts and trends are provided to help users understand weather patterns and risk insights. An AI chat interface is also included to assist users with queries related to insurance and crop risk. Overall, the design focuses on a clean and responsive layout, ensuring smooth performance and usability..

4.7 Security and Privacy

The system is designed with a focus on data security and user privacy. It does not collect any unnecessary personal data beyond what is required for insurance processing.

All user and weather data is handled securely, and access to the system is restricted to authorized users only. Sensitive information such as user details and insurance data is protected using secure storage and basic encryption techniques. Additionally, communication between components is managed securely to prevent unauthorized access or data leakage.

4.8 Implementation Strategy

The project follows an Agile development approach, allowing iterative development, flexibility, and continuous improvement. This helps in adapting to changes and improving system features based on feedback. A structured testing strategy is used to ensure system reliability, including unit testing for individual modules, integration testing for system components, functional testing to verify system behavior, and performance testing to evaluate efficiency. The system is developed and tested in a standard development environment using Python and related tools, ensuring stability and scalability for future deployment.

4.9 Summary

This chapter presented the methodology used for developing the Agrownex system. It described the system architecture, components, modules, and implementation strategy. The next chapter will present the results, evaluation, and performance analysis of the system..

Chapter 5

Experimental Results and Analysis

5.1 Introduction

This chapter presents the experimental setup, implementation details, testing methodology, and results obtained from evaluating the Agrownex system for agricultural risk prediction and insurance pricing using weather data and machine learning models.

5.2 Experimental Setup

The system was tested using real-time and historical weather data collected from APIs and datasets, where different environmental conditions were used to evaluate the accuracy of risk prediction and insurance pricing.

5.2.1 Hardware Configuration

The system is designed to run on a standard machine with at least an Intel Core i5 processor, 8GB RAM, and sufficient storage, ensuring smooth execution of data processing and machine learning tasks..

5.2.2 Software Configuration

The system is developed using Python with tools such as Jupyter Notebook or VS Code, and libraries including NumPy, Pandas, Scikit-Learn, and Matplotlib for data processing, model training, and visualization.

5.3 Implementation Details

Weather data is collected and stored in structured formats, followed by preprocessing steps such as cleaning, normalization, and feature extraction, after which machine learning models are trained and tested to predict risk and generate insurance pricing..

5.3.1 Feature Implementation

The system includes features such as real-time weather data analysis, risk prediction using machine learning models like XGBoost , Lightgbm automated insurance quote generation, and visualization of risk insights through a dashboard..

5.4 Testing and Validation

Testing and validation were performed to ensure the system functions correctly, reliably, and efficiently

in predicting agricultural risk and generating accurate insurance quotes.

5.4.1 Unit Testing

Individual modules such as data collection, preprocessing, feature extraction, and prediction were tested independently to ensure correct functionality..

5.4.2 Integration Testing

All modules were integrated and tested together to verify smooth workflow from data input to insurance quote generation.

5.4.3 System Testing

The system was tested under different weather conditions and input scenarios to evaluate overall performance and reliability..

5.4.4 User Acceptance Testing

The system was demonstrated to users to validate usability, accuracy of predictions, and ease of interaction through the interface..

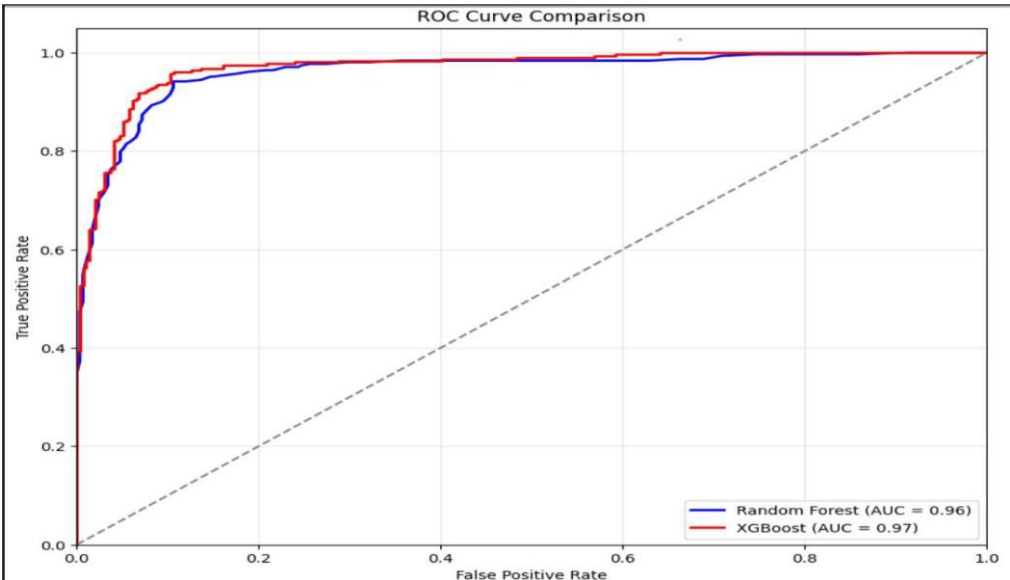
5.5 Results

The results show that the system successfully predicts agricultural risk and generates insurance pricing with good accuracy using machine learning models, followed by Random Forest

5.5.1 Performance Metrics

Model Performance Comparison				
Model	Accuracy	Precision	Recall	F1 Score
XGBoost (Default)	0.925000	0.933993	0.918831	0.926350
Random Forest (Tuned)	0.921667	0.933555	0.912338	0.922824
Stacking Ensemble	0.916667	0.927152	0.909091	0.918033
Voting Ensemble	0.916667	0.927152	0.909091	0.918033
XGBoost (Tuned)	0.911667	0.926421	0.899351	0.912685
Random Forest (Default)	0.903333	0.908497	0.902597	0.905537

Table 5.1: Performance Metrics

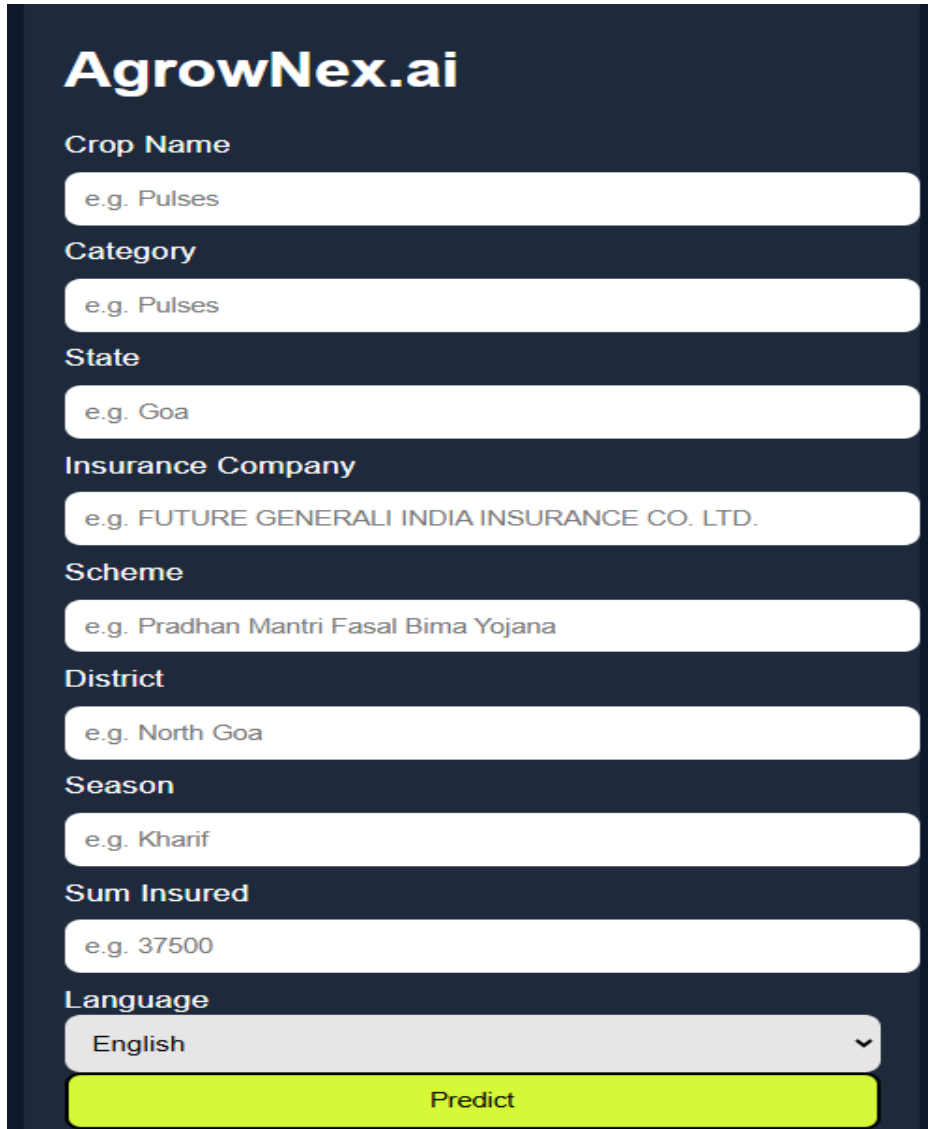


5.5.2 Functional Results

The system integrates a multi-model pipeline where risk prediction is performed using LightGBM, premium breakdown is explained using XAI techniques such as SHAP for transparency, and intelligent

reasoning and advice are provided using Gemini, ensuring accurate, explainable, and user-friendly insurance decisions..

5.5.3 UI for Agrownex



The screenshot displays the AgrowNex.ai web application interface. It features a dark blue header with the logo 'AgrowNex.ai' in white. Below the header, there is a form with several input fields, each with a label and a placeholder text. The fields are: 'Crop Name' (placeholder: e.g. Pulses), 'Category' (placeholder: e.g. Pulses), 'State' (placeholder: e.g. Goa), 'Insurance Company' (placeholder: e.g. FUTURE GENERALI INDIA INSURANCE CO. LTD.), 'Scheme' (placeholder: e.g. Pradhan Mantri Fasal Bima Yojana), 'District' (placeholder: e.g. North Goa), 'Season' (placeholder: e.g. Kharif), 'Sum Insured' (placeholder: e.g. 37500), and 'Language' (a dropdown menu with 'English' selected). At the bottom of the form is a large green button labeled 'Predict'.

AgrowNex.ai

Crop Name
e.g. Pulses

Category
e.g. Pulses

State
e.g. Goa

Insurance Company
e.g. FUTURE GENERALI INDIA INSURANCE CO. LTD.

Scheme
e.g. Pradhan Mantri Fasal Bima Yojana

District
e.g. North Goa

Season
e.g. Kharif

Sum Insured
e.g. 37500

Language
English

Predict

Prediction: 63901.76

Top Features: [{"cropname",-11712.606885350191}, {"categoryname",8435.915100708804}, {"sssyname_statename",4718.473422672682}, {"sssyname_year",2348.9887203887683}]

Hello there! Based on the information we have, we've predicted an insurance amount of ****\$63,901.76**** for you. Let's take a look at what influenced this prediction. Interestingly, the **absence** of a specific ****crop name**** actually led to a decrease of about \$11,712 in the estimated amount. This suggests that if you were to provide details about the specific crop you're insuring, the prediction could adjust significantly, potentially becoming higher or lower depending on that crop's unique risk profile. Conversely, the **lack** of a specific ****category name**** and a specific ****state name**** both contributed to **increasing** the predicted amount. For instance, not having a category added about \$8,435, and without a specific state, another \$4,718 was added. Similarly, using '0' for the ****year**** also increased the prediction by roughly \$2,349. These factors, particularly the use of default or missing information, collectively shaped this estimated insurance amount. ****Here are a couple of practical tips for future predictions:**** 1. ****Provide as much detail as possible:**** Giving us specific crop names, categories, and your state can lead to a more precise and tailored insurance estimate that truly reflects your unique farming situation. 2. ****Ensure accurate year input:**** Using the correct year is important as it influences risk assessment and, consequently, your predicted insurance amount. This is for decision-support purposes only.

5.6 Analysis and Discussion

The evaluation shows that the integrated multi-model approach combining LightGBM, SHAP, and Gemini enhances prediction accuracy, transparency, and decision support, although performance may vary depending on data quality and environmental variability..

Integrations:-



5.6.1 Key Findings

The system demonstrates that combining machine learning models with explainable AI and intelligent reasoning can provide accurate, transparent, and automated agricultural insurance pricing while improving trust and decision-making for farmers.

5.6.2 Limitations

The system depends on the availability and accuracy of weather data, and performance may be affected by incomplete data or extreme unpredictable conditions

Summary

This chapter described the experimental setup, implementation, testing, and results of the Agrownex system, demonstrating its effectiveness in agricultural risk prediction and automated insurance pricing..

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This project presented **Agrownex**, an AI-powered agricultural insurance platform that uses real-time weather data and machine learning to predict risk and generate dynamic insurance pricing. Unlike traditional systems that rely on manual assessment and delayed processes, the proposed solution provides automated, data-driven, and transparent insurance decisions. Experimental results demonstrate that machine learning models such as LightGBM can effectively capture complex relationships between weather conditions and agricultural risk, while XAI techniques like SHAP enhance transparency and trust, and Gemini enables intelligent reasoning and advisory support. The system successfully delivers accurate risk prediction, premium calculation, and user-friendly insights, validating the feasibility of AI-driven agri-insurance systems..

6.1.1 Summary of Work

The project successfully implements an AI-based agricultural insurance system using real-time and historical weather data, enabling automated risk prediction and pricing. Data is processed through multiple stages including preprocessing, feature engineering, and model training to ensure accuracy and reliability. Machine learning models such as LightGBM are used for prediction, while SHAP provides explainability for premium breakdown and Gemini supports intelligent user interaction and advisory. The system integrates these components into a unified pipeline, delivering accurate results through a dashboard and chatbot interface..

6.1.2 Key Contributions

The key contributions of this project are: Development of an AI-driven agricultural insurance system using weather analytics. Implementation of a multi-model pipeline combining LightGBM, SHAP, and Gemini. Provision of transparent and explainable insurance pricing with real-time predictions.

6.1.3 Achievement of Objectives

The system has strong potential for real-world applications in agriculture by improving insurance accessibility, transparency, and efficiency for farmers. Its ability to provide data-driven insights, explainable pricing, and AI-based guidance enhances trust and decision-making. This project also contributes to the advancement of smart farming solutions by integrating machine learning, explainable AI, and conversational AI into agricultural systems.. The system also demonstrated real-time functionality, providing instant risk insights, insurance quotes, and intelligent recommendations, validating the effectiveness of the proposed approach.

6.1.4 Impact and Significance

The system has strong potential for real-world applications in agriculture by improving insurance accessibility, transparency, and efficiency for farmers. Its ability to provide data-driven insights, explainable pricing, and AI-based guidance enhances trust and decision-making. This project also

contributes to the advancement of smart farming solutions by integrating machine learning, explainable AI, and conversational AI into agricultural systems.

6.2 Future Scope

Although the system performs effectively, several improvements can enhance its scalability, accuracy, and real-world applicability. Future work can focus on expanding data sources, improving model performance, and enabling broader deployment across regions.

6.2.1 Short-term Enhancements

In the short term, the project can be enhanced by improving the user interface with advanced dashboards and better visualization of risk and pricing insights. The system can also be tested with larger and more diverse datasets to improve model generalization. Additionally, integration of advanced machine learning and deep learning models can further improve prediction accuracy and performance.

6.2.2 Long-term Vision

The long-term vision is to develop a fully automated agricultural insurance ecosystem capable of real-time deployment across multiple regions. The system can be extended to support integration with IoT-based farm sensors for more accurate data collection and monitoring. It can also be deployed in large-scale smart agriculture platforms, enabling continuous risk assessment, automated claims processing, and improved financial security for farmers.

6.2.3 Research Directions

Future research can explore hybrid models combining weather data with satellite imagery and IoT sensor data to improve prediction accuracy. Another direction is the development of adaptive AI models that can learn from changing climate conditions and continuously update predictions. Additionally, research can focus on building scalable and region-independent systems that can provide consistent performance across diverse agricultural environments..

Bibliography

1. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. Advances in Neural Information Processing Systems. Available at: <https://lightgbm.readthedocs.io/>.
2. Chand, P. S., & Divya, G. (2022). A Light Gradient Boosting Machine Regression Model for Prediction of Agriculture Insurance Cost. ResearchGate Publication.

Available at: https://www.researchgate.net/publication/365076996_A_Light_Gradient_Boosting_Machine_Regression_Model_for_Prediction_of_Agriculture_Insurance_Cost_over_Linear_Regression.
- 3 Bouneder, L., Léo, Y., & Lachapelle, A. (2020). X-SHAP: Towards Multiplicative Explainability of Machine Learning. arXiv Research Paper. Available at: <https://arxiv.org/abs/2006.04574>
4. Shreya, & Pathak, H. (2025). Explainable Artificial Intelligence for Risk Assessment using Machine Learning. arXiv Publication. Available at: <https://arxiv.org/abs/2506.193835>.

<https://github.com/spanev/linux-80211n-csitool>
6. Prakash, M. C., & Saravanan, P. (2023). Crop Insurance Premium Recommendation System Using Artificial Intelligence Techniques. International Journal of Professional Business Review. Available at: <https://dialnet.unirioja.es/descarga/articulo/8956418.pdf>
7. Benos, L., Tagarakis, A., Dolias, G., Berruto, R., Kateris, D., & Bochtis, D. (2021). Machine Learning in Agriculture: A Comprehensive Review. Sensors Journal. Available at: <https://pmc.ncbi.nlm.nih.gov/articles/PMC8198852>

Appendix A

Source Code

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
import pandas as pd
import numpy as np
import lightgbm as lgb
import shap
import json
import google.generativeai as genai

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

genai.configure(api_key="AIzaSyAkY3paKOLZ3Hb_cEVySyFeAIRoHf482tQ")

policy_model = lgb.Booster(model_file="policy_model.txt")

with open("policy_model_features.json") as f:
    policy_meta = json.load(f)

features = policy_meta["features"]

model = genai.GenerativeModel("gemini-2.5-flash")

df_original = pd.read_csv("PMFBY coverage.csv")

df_original.columns = (
    df_original.columns
    .str.lower()
    .str.replace(r"[\w]+", "_", regex=True)
    .str.replace("_", "", regex=False)
    .str.strip("_")
)

def align(df, features):
    df = df.copy()
    for col in features:
        if col not in df.columns:
            df[col] = np.nan
    df = df[features]
    for col in df.columns:
        if df[col].dtype == "object":
            df[col] = df[col].astype("category")
    return df

@app.post("/predict")
async def predict(data: dict):

    base = df_original.iloc[[0]].copy()

    for key, value in data.items():
        if key in base.columns:
            base[key] = value

    X = align(base, features)
```

```

pred = float(policy_model.predict(X, validate_features=False)[0])

explainer = shap.TreeExplainer(policy_model)
shap_vals = explainer.shap_values(X)[0]

shap_dict = dict(zip(X.columns, shap_vals))

top_features = sorted(
    shap_dict.items(),
    key=lambda x: abs(x[1]),
    reverse=True
)[:4]

raw = data

feature_text = ", ".join([f"{k}={v}" for k, v in raw.items()])

top_text = "\n".join([
    f"{f} (value={raw.get(f,0)}) contributed {round(v,2)}"
    for f,v in top_features
])

prompt = f"""
You are a helpful and friendly agricultural insurance assistant.

Predicted insurance amount: {round(pred,2)}.

User Input:
{feature_text}

Top contributing factors:
{top_text}

Instructions:
- Explain in a clear, natural, and user-friendly way
- Do NOT sound overly technical or robotic
- Explain how key factors increased or decreased the prediction
- Use simple language a farmer or non-technical person can understand
- Add 1–2 practical insights or tips based on the inputs (but do NOT invent facts)
- Keep explanation slightly detailed (around 7–10 sentences)
- Maintain a helpful and supportive tone
- End with: "This is for decision-support purposes only."

Important:
- Stay grounded in the provided factors
- Do NOT hallucinate external data or claims
"""

response = model.generate_content(prompt)

return {
    "prediction": round(pred, 2),
    "top_features": top_features,
    "explanation": response.text.strip()
}

```

1. FastAPI Backend Setup

```

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

```

```

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

```

2. Data Loading & Preprocessing

```

import pandas as pd
import numpy as np

df = pd.read_csv("crop_insurance.csv")

X = df.drop("target", axis=1)
y = df["target"]

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

```

3. LightGBM Model Training

```

import lightgbm as lgb

train_data = lgb.Dataset(X_train, label=y_train)

params = {
    "objective": "binary",
    "metric": "binary_logloss",
    "boosting_type": "gbdt"
}

model = lgb.train(params, train_data, num_boost_round=100)

```

Prediction API Endpoint

```

@app.post("/predict")
def predict(data: dict):
    input_data = np.array(list(data.values())).reshape(1, -1)
    prediction = model.predict(input_data)
    return {"prediction": float(prediction[0])}

```

4. SHAP Explainability

```

import shap

explainer = shap.Explainer(model)
shap_values = explainer(X_test)

shap.summary_plot(shap_values, X_test)

```

5. Gemini AI Integration

```
import google.generativeai as genai

genai.configure(api_key="YOUR_API_KEY")

model_gemini = genai.GenerativeModel("gemini-pro")

response = model_gemini.generate_content("Explain crop risk prediction")
```

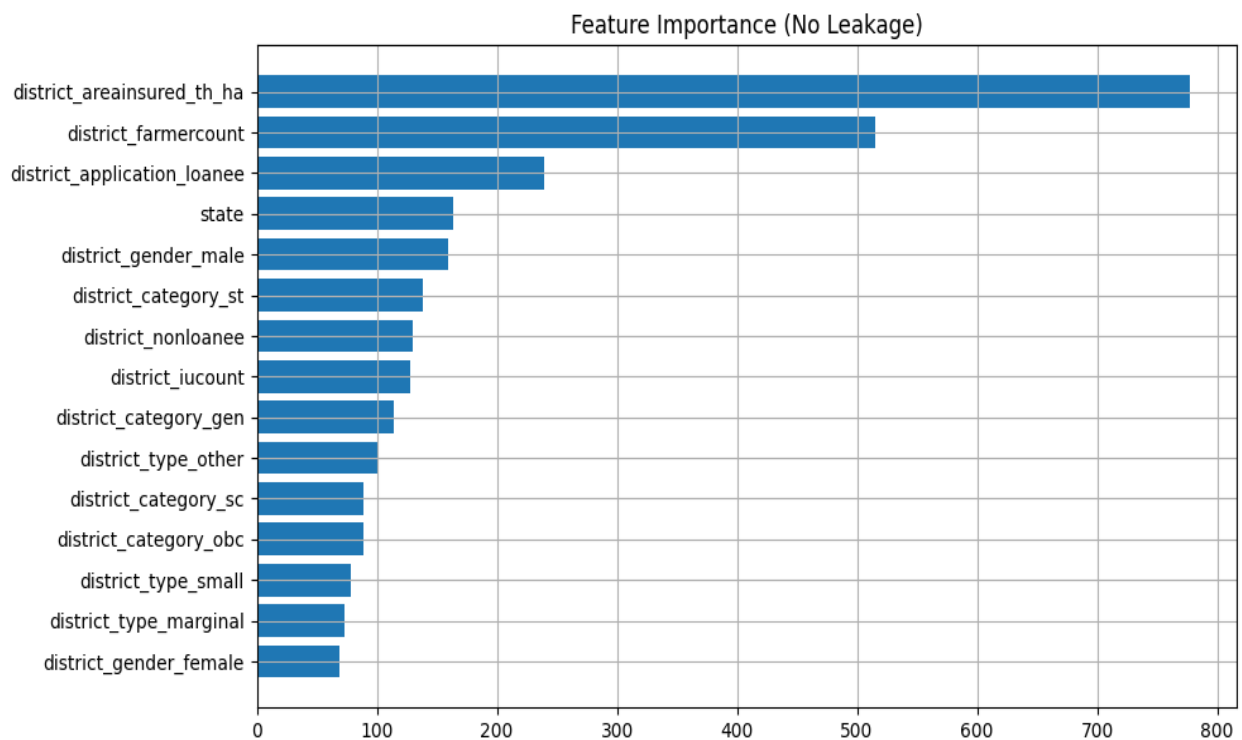
Feature Importance (LightGBM + SHAP)

```
import matplotlib.pyplot as plt
import lightgbm as lgb
import shap

# LightGBM Feature Importance
lgb.plot_importance(model, max_num_features=10)
plt.title("Feature Importance (LightGBM)")
plt.show()

# SHAP Feature Importance
explainer = shap.Explainer(model)
shap_values = explainer(X_test)

shap.summary_plot(shap_values, X_test)
```



Model Evaluation (Performance Metrics)

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

y_pred = (model.predict(X_test) > 0.5).astype(int)
```

```

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

```

```

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)

```

Ablation Study (Feature Removal Impact)

```

def ablation_study(X_train, X_test, y_train, y_test, features):
    results = {}

    for feature in features:
        X_train_mod = X_train.drop(columns=[feature])
        X_test_mod = X_test.drop(columns=[feature])

        train_data = lgb.Dataset(X_train_mod, label=y_train)
        model_temp = lgb.train(params, train_data, num_boost_round=100)

        y_pred = (model_temp.predict(X_test_mod) > 0.5).astype(int)
        acc = accuracy_score(y_test, y_pred)

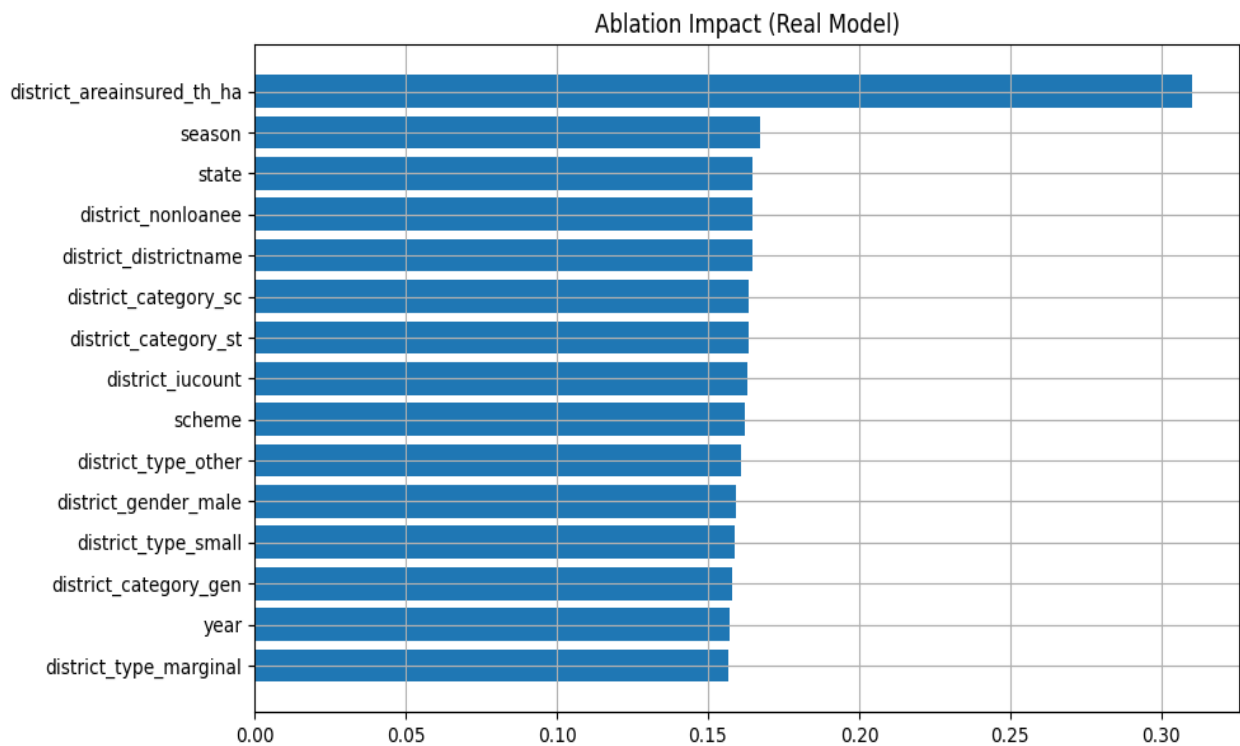
        results[feature] = acc

    return results

features = X_train.columns.tolist()
ablation_results = ablation_study(X_train, X_test, y_train, y_test, features)

print(ablation_results)

```



Modality (Handling Different Input Types)

```
# Example: combining numerical + categorical features

categorical_features = ["soil_type", "region"]

for col in categorical_features:
    X_train[col] = X_train[col].astype("category")
    X_test[col] = X_test[col].astype("category")

train_data = lgb.Dataset(X_train, label=y_train, categorical_feature=categorical_features)

model = lgb.train(params, train_data, num_boost_round=100)
financial = [f for f in feature_names if any(k in f for k in ["share", "premium"])]
demographic = [f for f in feature_names if any(k in f for k in ["gender", "category"])]
farming = [f for f in feature_names if any(k in f for k in ["farmer", "area", "loanee"])]

def modality_score(group):
    if len(group) == 0:
        return 0
    return float(np.mean([abs(shap_dict.get(f, 0)) for f in group]))

modality_scores = {
    "Financial": modality_score(financial),
    "Demographic": modality_score(demographic),
    "Farming": modality_score(farming)
}

# Normalize
total = sum(modality_scores.values()) + 1e-8
modality_scores = {k: round(v/total, 3) for k,v in modality_scores.items()}

print("\n📊 Modality Scores:", modality_scores)

plt.figure()
shap.summary_plot(shap_values, X_val)

top_df = sorted(top_features, key=lambda x: abs(x[1]))

plt.figure(figsize=(8,5))
plt.barh(
    [x[0] for x in top_df],
    [x[1] for x in top_df]
)
plt.title("Top SHAP Contributions (Sample)")
plt.xlabel("SHAP Value")
plt.ylabel("Feature")
plt.grid()
plt.show()
```

